Project 2: The Librarian

CS496 AI Engineering, Fall 2026. A retrieval-augmented question answering service over Tunisian laws, in Arabic and French, with citations. Draft of 3 Nov 2026. Sections marked **[TO FILL]** depend on runs that are not done yet (Week 8); nothing in them is estimated.

At a glance. 171 Arabic and 179 French laws (2013-2026), 15,831 article-level chunks, searched with keyword + vector + reranker. On our questions the right article is in the top 5 for 100% of the main set and 76% of the paraphrased set; answers are 95-96% correct, 96% faithful to their citations, and 23 of 24 unanswerable questions are refused (22-23 of 24, decided by the model; a score threshold does not generalise, section 4). A request takes about 5 s and costs $0.0002. Not done: real-user feedback; Week 8 numbers are pending.

1. The corpus and its users

Corpus. Laws published in the *Journal Officiel de la République Tunisienne* (Imprimerie Officielle, iort.gov.tn), 2013 to 2026: **171 Arabic texts and 179 French texts** (134 laws exist in both languages), about 12 million characters, scraped with `src/ingest/iort_scrape.py`. It is real (the legal texts in force), public (no permission needed, no personal data), larger than any prompt, and every fact can be checked against an article. Loan and treaty ratifications were left out on purpose (about 450 of the 635 laws the site lists for 2013 to 2026); budget and finance laws are the biggest files and are mostly tables. Texts before 2013 could not be used: their PDFs use legacy fonts and extract as junk.

Users. Law students, lawyers, journalists and citizens who want the text of a rule, a delay, an authority or a penalty, in either language. The questions the system must handle look like *"in how many days must the Instance announce final election results?"* or *"can a candidate excluded by the parliamentary commission appeal?"*. The service has thumbs up/down and logs ready for user feedback, but **we did not collect feedback from real users**: every number in this report comes from our own question sets.

Evaluation data. 124 questions (`data/eval/qa.jsonl`): 50 facts, each asked in Arabic and in French, plus 12 questions the corpus cannot answer, in both languages. Each answerable question has a reference answer, the gold law and article, and the evidence passage. A second set of 50 **paraphrased** questions (25 facts x 2 languages, `qa_hard.jsonl`) avoids the laws' own wording and law numbers: the first set flatters keyword search, the second is the honest test. All were written by reading the French and Arabic articles side by side.

2. Pipeline design

Stage	Choice	Why
Chunking	One chunk per legal article (الفصل / Article), tiny articles merged (<200 chars), long ones split at 1,200 chars with 150 overlap; each chunk starts with "law reference, article"	A citation must name a law and an article. 15,831 chunks, median 1,001 chars
Index	Flat files: `embeddings.npy` (15,831 x 1,536, float16, 47 MB) + token lists, brute-force cosine	At 16k chunks a matrix product is milliseconds; a vector database would be one more service and no accuracy gain
Keyword search	BM25 with Arabic normalisation (alef variants, diacritics, tatweel, definite article) and French accent stripping	Legal questions often quote numbers and names
Vector search	OpenAI `text-embedding-3-small` (index built in 166 s for $0.135); local `multilingual-e5-small` as comparison (57 min on CPU)	Multilingual, so a French question finds the Arabic article and vice versa
Fusion	Reciprocal rank fusion (k=60) of the two top-30 lists	No score calibration needed
Reranking	Cross-encoder `mmarco-mMiniLMv2` on the top 30	Largest single accuracy gain, and the largest cost in time
Generation	`gpt-4o-mini`, temperature 0, answers only from the 5 numbered chunks, a citation `[n]` after every claim, reply in the question's language, "I don't know" otherwise	Faithfulness over fluency
Abstention	The model's instruction: "I don't know" when the sources lack the answer. A refusal threshold on the best reranker score was tested (2.0) and **is not used in the deployed service**	It separates the main-set questions perfectly but would wrongly refuse 74% of paraphrased ones (section 4)

3. Retrieval results and the ablation

A hit means the retrieved chunk is from the gold law and contains the gold article (matched by article number or evidence text, so Arabic and French chunks both count). Index: article chunking, OpenAI embeddings. With n = 100 (main) and n = 50 (paraphrased), differences under about 8 points are within noise.

search	main recall@1	main recall@5	main MRR	paraphrased recall@1	paraphrased recall@5	paraphrased MRR
keyword only (BM25)	87%	93%	0.898	12%	32%	0.207
vector only	73%	92%	0.817	34%	66%	0.481
hybrid (RRF)	84%	94%	0.894	26%	58%	0.386
hybrid + reranker	**97%**	**100%**	**0.983**	**52%**	**76%**	**0.612**

What each part added. On the main set the reranker lifts recall@1 from 84% to 97% and recall@5 to 100%; hybrid search adds little over BM25 alone because the questions reuse the articles' words. On the paraphrased set keyword search collapses (32%), the vector search carries the system (66%), and **hybrid is worse than vector alone** (58%): BM25 pushes wrong articles into the fused list. The reranker recovers most of it (76%), but recall@10 is also only 78%, so the remaining misses never reach the reranker: the candidate search is the limit there.

Chunking (e5 embeddings, recall@1 / MRR): article chunks beat 1,200-character fixed windows at the top ranks on both sets, for BM25 (87% vs 78%), vector (73% vs 59%) and the reranked system (97% vs 94% main; 52% vs 46% paraphrased). Recall@5 is similar (the right text is nearby either way), so article chunking mostly helps put the right article first, and it makes citations exact.

Embeddings. OpenAI and local e5 are close overall (reranked recall@5 on paraphrased questions: 76% vs 74%). They differ by language: vector-only recall@5 on paraphrased questions is 88% (French) and 44% (Arabic) with OpenAI against 60% and 68% with e5. With 25 facts per language this is a hint, not a result. The OpenAI index is what is deployed: it builds in 3 minutes instead of an hour and the service needs no local embedding model.

Latency. BM25 66 ms, vector 150 ms (e5, local) to 330 ms (OpenAI, one API call), hybrid 160-490 ms, and **hybrid + reranker 7-9 s per query** on this 8-core CPU, which dominates the whole request (section 5).

4. Generation results

124 questions (100 answerable, 24 not), generator `gpt-4o-mini`, judge `gpt-4.1-mini`, same pipeline as section 3 (hybrid + reranker, 5 chunks, OpenAI-embedding index). Correctness is judged against the reference answer, faithfulness against the chunks the answer cites, and "cited" means the gold article is among them.

	baseline	+ prompt fix, threshold 2.0	same run, faithfulness judge fixed
Answer correctness	87%	96%	95%
Faithfulness (claims supported by the cited chunks)	85% *	83% *	**96%**
Gold article cited	99%	99%	99%
Citation precision (cited chunks that are gold)	96%	91%	91%
Unanswerable questions refused	22 of 24	23 of 24	23 of 24
Answerable questions refused wrongly	0	0	0
Latency p50 / p95	4.9 s / 5.8 s	4.9 s / 6.1 s	not comparable (other machine)
Cost per request	$0.00021	$0.00019	$0.00019

* measured with a faithfulness judge that numbered the sources 1..k while the answer cites their original positions (`[3][4]`), so answers were checked against the wrong chunks. After the fix, faithfulness is 96% (Arabic 98%, French 94%); correctness by language is 96% (Arabic) and 94% (French).

What the baseline failures were, and what fixed them. Of the 13 wrong answers in the baseline, 11 were incomplete, not false: the answer gave the main fact and dropped the doubled fine in case of repeat offence, the breakdown into expenses and surplus, or the register where identities are recorded. Three French questions were answered in Arabic, including 9 of 12 French refusals. One prompt change (give the complete answer; answer in the question's language even when the sources are in the other) took correctness to 95-96% (the threshold refused no answerable question on this set, so it did not move correctness). The unanswerable questions the baseline answered were the petrol price, in both languages: it quoted fuel duties (5 millimes per litre, 41.382 dinars per hectolitre) as if they were the price.

What still fails (5 of 100 answerable questions wrong; the run-to-run difference of one point is noise): three omissions (two budget figures without their breakdown, one register detail); one over-inclusion (extra conditions from another law, a side effect of asking for complete answers); and one real error, a French question on how civil senior posts are filled, answered "by government decree" where the law says "by order of the head of government": the gold article was among the retrieved chunks but the answer was built on another. The only unanswerable question not refused (minimum wage, French) was answered with "the amount is not specified in the sources", which is a refusal in substance but is counted as an answer because it cites a related law.

The refusal threshold does not generalise. The 23-of-24 above includes a refusal when the best reranker score is below 2.0. On the main set that costs nothing (0 of 100 answerable questions refused) and refuses 21 of the 24 unanswerable ones by score alone. On the paraphrased questions it would wrongly refuse **74% (37 of 50)** of answerable ones, and even a threshold of 0 would refuse 40% (`results/threshold_check.md`): the cross-encoder gives high scores when the question reuses the article's words and low ones when it does not, so its score measures wording overlap, not whether the answer exists. The threshold only looked good because the main set reuses the laws' wording. The deployed service therefore leaves it off and relies on the model's own refusal (22 of 24 in the baseline).

Limits. One run, n = 100, an LLM judge, reference answers that reward completeness, and no real-user feedback. These are numbers on the main set, whose questions reuse the laws' wording; the paraphrased questions have not been run through generation (retrieval only: 76% recall@5), so answer quality there is unknown and probably lower.

5. Week 8: what changed and what it saved [TO FILL by DF: every cell marked [DF]]

Target. Cut p95 latency or cost per request by 50% while losing at most 2 points of accuracy, measured before and after on the same 124 questions, on one machine with nothing else running. Accuracy here is answer correctness (95-96% before); faithfulness and the refusals on unanswerable questions are reported alongside.

Where the time and money go. The reranker is almost all of the latency (about 7 s of the 8-9 s request on a slow machine; p50 4.9 s and p95 6.1 s on DF's). Generation costs $0.0002 per request, so latency is the larger target; the cost levers are a cheaper model, fewer chunks and caching.

Each change is measured alone first, then the kept ones together (all switches are off by default):

#	change	how	p50	p95	cost per request	correctness	faithfulness	unanswerable refused	kept?
0	baseline (prompt fix, threshold 2.0)	none	4.9 s	6.1 s	$0.00019	96%	96%	23/24	
1	fewer rerank candidates	`--n-candidates 10`	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]
2	shorter rerank input	`--rerank-max-length 256`	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]

#	change	how	p50	p95	cost per request	correctness	faithfulness	unanswerable refused	kept?
3	skip the reranker when the first search is confident	`--skip-rerank-gap` [DF: value]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]
4	fewer chunks sent to the model	`--k 3`	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]
5	cheaper generation model	`--model` [DF: e.g. `gpt-5-nano`]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]
6	cache repeated questions	`CACHE_ANSWERS=1`	[DF]	[DF]	[DF]	n/a	n/a	n/a	[DF]
final	the kept changes together	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	[DF]	

Result. [DF: one sentence, for example "p95 latency fell from 6.1 s to X s (-Y%) and cost per request from $0.00019 to $Z (-W%), with correctness 96% to V% (-U points)", and whether the 50% target and the 2-point limit were met.]

What did not work [DF: changes that lost more than 2 points or saved too little, with the numbers, and why]. One thing to check when judging a change: the skip-reranker option returns fusion scores, not reranker scores. The refusal threshold is not used in the deployed service (section 4), so this only matters if one is added back.

6. Known failure cases and cost model

Failure cases (all seen in this project):

- **Amendments are not merged.** The corpus holds each law as published. Asking for the delay in which election results are announced returned the 2024 amending law (Court of Cassation, presidential elections), not the 2014 law the question was written from. The answer is faithful to its source, but the user may not realise the article was amended.
- **Tables.** Budget and finance laws extract as loose lines of numbers; questions about a figure in a table are not answerable reliably, and these files dominate the chunk count.
- **Parallel texts disagree.** Article numbers in an *amending* law can differ between the Arabic and French versions, and some Arabic files contain the end of a neighbouring law (the site's PDFs span gazette pages).
- **Near-identical laws.** Several mutuelle laws differ only in a minister's name; they are found by the law number in the question, not by meaning.
- **Paraphrased questions** lose most of keyword search, and Arabic vector search looked weaker than French with the OpenAI embeddings.
- **Coverage.** 2013-2026 only, none for 2022, and the Arabic side is thinner than the French.
- **Latency.** The reranker makes every request 5-9 s on CPU (about 5 s on DF's machine, 7-9 s on a slower one).

Cost model (`eval/cost_model.py`, computed from the generation baseline; full table in `results/cost_model.md`; list prices to be re-checked):

Item	Value
Index build, once	$0.135 (6.76 M tokens), 166 s
Question embedding	about 25 tokens: $0.0000005 per request
Generation (`gpt-4o-mini`, 5 chunks, about 1,000 prompt tokens)	$0.00021 per request (measured on the 124 baseline requests)
Per request	**about $0.0002, 4.9 s p50 / 5.8 s p95** (124 requests, DF's machine)

Item	Value
Today: 100 requests per day (a class demo)	$0.63 per month in model fees; free CPU Space, 1 worker
At 100 times: 10,000 requests per day	$63 per month in model fees; 3 workers at 25% utilisation, about $90 per month if a worker costs $30 (an assumed price)

Hosting grows with the reranker's CPU time, not with model fees, which is why it is the Week 8 target.

[TO FILL] after the Week 8 changes: p50/p95 latency and cost before and after.